Alerta Rio: monitoramento comunitário de enchentes

Global Solution FIAP · CitizenTech · Coding for Security | Aluno: Diego Araujo Silva RM: 573684 | Novembro/2026
Repositório: https://github.com/DiegoSilva77/citizentech-alerta-rio

1. Problema, cidadão e evidência

Problema: em áreas ribeirinhas, o aviso de enchente chega tarde ou de forma genérica ("risco alto no estado"). O morador não sabe **qual arroio** está subindo, **quanto tempo** tem e **para onde ir**. **Evidência (dado público):** nas enchentes de maio de 2024 no Rio Grande do Sul, o boletim da Defesa Civil RS de 03/06/2024 registrou **475 municípios afetados, cerca de 2,39 milhões de pessoas atingidas, 579 mil desalojados e 172 mortes**. Relatos da imprensa e de moradores mostraram famílias surpreendidas pela água dentro de casa e pessoas que se recusaram a sair por não poderem levar seus animais.

Cidadão beneficiado: famílias que moram perto de arroios, rios e encostas, sobretudo idosos, pessoas com deficiência e tutores de animais. **Ecossistema:** moradores (consultam o painel), lideranças comunitárias e voluntários (instalam e mantêm os sensores nas pontes), Defesa Civil (operador que cadastra sensores e abrigos), abrigos como escolas, ginásios e igrejas (informam vagas, acessibilidade e se aceitam pets) e a Open-Meteo (previsão de chuva aberta).

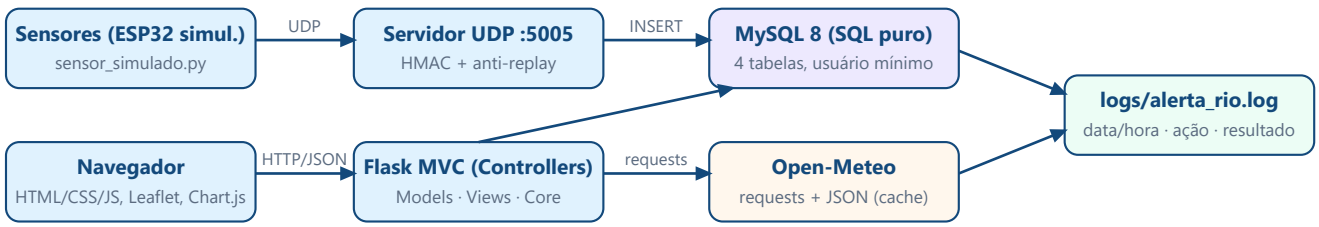
2. Objetivos de Desenvolvimento Sustentável

ODS	Meta	Como o Alerta Rio contribui
11 · Cidades e comunidades sustentáveis	11.5: reduzir mortes e pessoas afetadas por catástrofes, inclusive as relacionadas à água	Alerta local e antecipado por ponto do rio; mapa de abrigos com vagas livres, acessibilidade e aceitação de pets
13 · Ação contra a mudança do clima	13.1: reforçar a resiliência e a capacidade de adaptação a riscos climáticos	Cruza nível do rio com a previsão de chuva; histórico de leituras e alertas que serve de base para cobrar obras e planejar a resposta

3. Como a solução funciona

Sensores de nível (simulados em Python, representando ESP32 + sensor ultrassônico) enviam leituras **assinadas** por **socket UDP**. O servidor UDP valida o pacote, grava a leitura no MySQL e recalcula o risco. Quando o risco muda, registra um alerta. A API Flask entrega ao painel web o risco de cada ponto, a previsão da Open-Meteo, o mapa de abrigos e o histórico. A regra é transparente: nível ≥ crítico → **PERIGO**; nível ≥ alerta → **ATENÇÃO** (vira PERIGO com ≥ 10 mm de chuva prevista em 6 h); nível ≥ 70% do alerta com ≥ 25 mm em 6 h → ATENÇÃO; caso contrário → **NORMAL**. A tela de Gestão permite à Defesa Civil fazer o CRUD de sensores e abrigos com chave de operador.

4. Arquitetura e tecnologias



Requisito	Implementação
API REST Flask (GET/POST/PUT/DELETE)	`/api/sensores` e `/api/abrigos` com 200, 201 (+Location), 204, 400, 401, 404, 405, 409, 413, 415, 429 e 503; também /resumo, /clima, /alertas e /leituras
MySQL sem ORM	mysql-connector-python com pool; SQL puro e parametrizado na camada Model (models/)
Fonte externa (requests + JSON)	Open-Meteo: temperatura, umidade, chuva/hora e probabilidade; timeout 6 s, cache 10 min
Interface web	HTML/CSS/JS puro + Leaflet (mapa OSM), Chart.js (gráficos), Lucide (ícones), Plus Jakarta Sans
Log em arquivo	logs/alerta_rio.log: data e hora · origem · ação · resultado · detalhe (rotação a cada 1 MB)
Módulo escolhido: sockets	UDP (leve para dispositivos de campo) com ACK, limite de 1 KB e tolerância a pacotes ruins
Organização: padrão **MVC**	**Model** (models/): SQL, Open-Meteo e regra de risco, sem Flask. **View** (views/): HTML/CSS/JS e respostas JSON padronizadas. **Controller** (controllers/): um Blueprint por recurso, além do controller do pacote UDP e o de erros. core/: validação, segurança e log. Exceções próprias (ErroBanco, ErroValidacao, ClimaIndisponivel)

5. Modelo de dados

Tabela	Campos	Regras
sensores	id, codigo, nome, bairro, latitude, longitude, nivel_alerta_cm, nivel_critico_cm, ativo, criado_em, atualizado_em	codigo UNIQUE; CHECK crítico > alerta
leituras	id, sensor_id (FK), nivel_cm, registrado_em	índice (sensor_id, data); ON DELETE CASCADE
alertas	id, sensor_id (FK), nivel_risco ENUM(normal, atencao, perigo), nivel_cm, mensagem, criado_em	gravado só quando o risco muda

Tabela	Campos	Regras
abrigos	id, nome, endereco, bairro, latitude, longitude, capacidade, ocupacao, aceita_pets, acessivel, ativo	CHECK ocupação ≤ capacidade

Relacionamentos: sensores 1:N leituras e sensores 1:N alertas. **LGPD:** não existe tabela de pessoas; o sistema funciona sem identificar o morador.

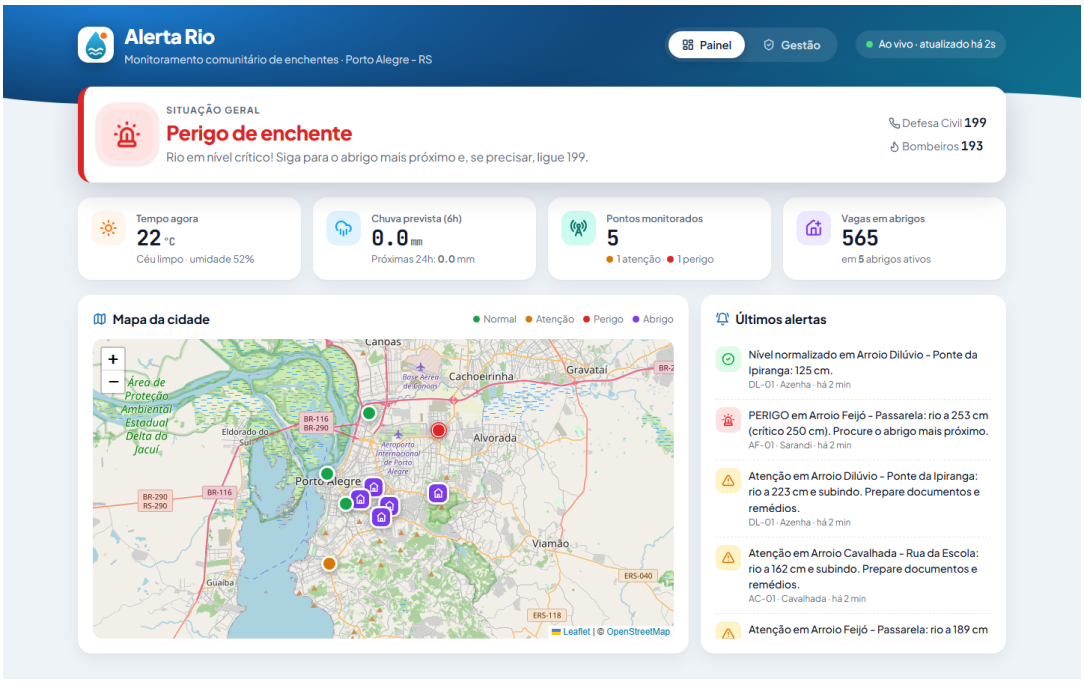


Figura 1: painel web em cenário de enchente simulada (dados fictícios, mapa © OpenStreetMap).

6. Segurança e privacidade

Controles obrigatórios: credenciais e segredos no .env (fora do Git, com .env.example); 100% das consultas com parâmetros %s; validação de toda entrada (tipo, tamanho, regex, faixa geográfica do Brasil, regras de negócio e recusa de campos desconhecidos contra mass assignment); erros genéricos ao cliente com detalhe técnico só no log; coleta mínima: nenhum dado pessoal, e o log não guarda IP nem chaves. **Extras:** usuário de banco com privilégio mínimo (só SELECT, INSERT, UPDATE e DELETE), cabeçalhos CSP, X-Frame-Options e nosniff, corpo máximo de 16 KB e front-end que só usa textContent.

Ameaça	Impacto	Mitigação implementada
1. Falsificação de leituras do sensor (spoofing/replay via UDP)	Alarme falso que gera pânico ou, pior, uma enchente real escondida	Assinatura HMAC-SHA256 com segredo do .env e comparação em tempo constante; janela de 30 s e memória de assinaturas (anti-replay); sensor precisa estar cadastrado e ativo; validação e limite de 1 KB por pacote
2. Alteração não autorizada de dados (força bruta na API de escrita)	Abrigo apagado ou com vagas falsas, sensor desativado	Chave de operador (X-API-Key) com hmac.compare_digest; rate limit de 30 escritas/min e 10 tentativas/min de login; tentativa negada registrada no log; chave guardada só no sessionStorage
3. Injeção (SQL Injection e XSS armazenado)	Vazamento ou destruição do banco; script malicioso no navegador do morador	Queries parametrizadas; regex que bloqueia < > ; { }; usuário do banco sem DROP/ALTER; DOM montado com textContent; Content-Security-Policy script-src 'self'

7. Validação e viabilidade

Testes automatizados (pytest): 37 aprovados, cobrindo a separação MVC (models sem Flask, controllers sem SQL), códigos HTTP, autenticação, cabeçalhos de segurança, entradas maliciosas (XSS, SQLi, coordenadas fora do Brasil, booleano disfarçado de número), regra de risco, HMAC e anti-replay. **Teste ponta a ponta:** CRUD real no MySQL com todos os status esperados; cenário enchente gerou alertas de ATENÇÃO e PERIGO em tempo real; o modo --invasor teve o pacote com assinatura falsa recusado ("não autorizado") e o reenvio de um pacote legítimo recusado ("pacote expirado"); a Open-Meteo respondeu ao vivo e, se cair, a API devolve 503 sem quebrar o painel. **Viabilidade:** Python, MySQL, Open-Meteo e OpenStreetMap são gratuitos. Cada ponto real exigiria um microcontrolador com sensor ultrassônico à prova d'água e alimentação solar, um hardware de baixo custo que pode ser mantido pela própria comunidade (o valor exato depende de cotação). A API roda num servidor pequeno.

8. Limitações e próximos passos

Sensores simulados; limiares fictícios (no mundo real viriam das cotas históricas da Defesa Civil e da ANA); sem notificação ativa (próximo passo: SMS ou WhatsApp com opt-in e o mínimo de dados); chave de operador única (ideal: login individual com perfis); execução local em HTTP (produção: HTTPS, Redis para o rate limit, backup do banco); não há validação com moradores reais, que seria o próximo passo antes de um piloto.

9. Recursos além do conteúdo (bônus) e por que foram usados

Recurso	Justificativa
HMAC-SHA256 + anti-replay	UDP não autentica a origem. Sem assinatura, qualquer um na rede forjaria o nível do rio
Flask-Limiter (rate limiting)	Freia força bruta na chave de operador e o abuso da API
CSP e cabeçalhos de segurança	Segunda camada contra XSS e clickjacking, além da validação
Pool de conexões MySQL	API e servidor UDP acessam o banco ao mesmo tempo sem abrir conexão a cada requisição
Leaflet/OSM, Chart.js, Lucide e Rich	Mapa e gráfico tornam o risco compreensível num relance; Rich deixa a demo do socket legível
pytest com monkeypatch	Comprova segurança e regras sem depender do banco: qualquer avaliador consegue rodar

Referências: Defesa Civil RS, boletim de 03/06/2024 (estado.rs.gov.br); ONU Brasil, Objetivos de Desenvolvimento Sustentável (brasil.un.org/pt-br/sdgs); Open-Meteo Weather API (open-meteo.com); OpenStreetMap (openstreetmap.org); Lei 13.709/2018 (LGPD). Todos os sensores, abrigos e leituras do protótipo são fictícios.